

Uso de álgebras modernas para seguridad y criptografía

1 El algoritmo de Euclides

Definición 1.1. Supongamos que $n > m \geq 0$. Decimos que d es un *máximo común divisor* de n y m si se satisfacen las siguientes condiciones:

1. $d > 0$.
2. $r|n$ y $r|m$.
3. Si $d|n$ y $d|m$, entonces $d|r$.

Ejemplo 1.2. Consideremos los siguientes ejemplos:

1. Supongamos que

$$n = 6 = 2 \cdot 3, \quad m = 4 = 2^2.$$

En este caso, si tomamos $d = 2$, vemos que d es un máximo común divisor de 6 y 4.

2. Si

$$n = 36 = 2^2 \cdot 3^2, \quad m = 24 = 2^3 \cdot 3,$$

entonces

$$r = 2^2 \cdot 3 = 12$$

es un máximo común divisor de 36 y 24.

Teorema 1.3 (Algoritmo de Euclides). *Supongamos que $n > m \geq 0$. Entonces:*

1. *Existe $d \in \mathbb{Z}$ tal que d es un máximo común divisor de n y m .*
2. *Existen $a, b \in \mathbb{Z}$ tal que*

$$an + bm = d.$$

Proof. Para la demostración de este resultado, usaremos la interpretación BHK (Brouwer-Heyting-Kolmogorov) que nos indica que la demostración de un teorema de existencia debe tener la siguiente estructura:

1. Debemos presentar un *algoritmo* que nos permita calcular los número r , a y b a partir del par de enteros $n > m \geq 0$.
2. Debemos demostrar que los números r , a y b obtenidos en el algoritmo anterior satisfacen las propiedades requeridas.

Empezaremos por indicar un algoritmo para calcular el máximo común divisor, d , de $n > m \geq 0$. Para esto definiremos una función recursiva $d = \text{mcd}(n, m)$ de la siguiente manera:

1. Si $m = 0$, entonces definimos

$$d = \text{mcd}(n, m) = n.$$

2. Si $m \neq 0$, entonces, usando el algoritmo de la división, encontramos q, r con $0 \leq r < m$ tales que

$$n = qm + r.$$

En este caso definimos

$$d = \text{mcd}(m, r),$$

donde para calcular $\text{mcd}(m, r)$ usamos el algoritmo anterior de manera recursiva hasta encontrar el resultado.

Antes de continuar con la demostración, consideraremos unos ejemplos para ilustrar el funcionamiento del algoritmo dado. Usando los valores del ejemplo 1.2 obtenemos:

1. Cuando $n = 6$, y $m = 4$ y aplicamos nuestro algoritmo, observamos que, como $m \neq 0$, debemos usar el algoritmo de la división para calcular

$$d = \text{mcd}(6, 4) \quad 6 = 4(1) + 2, \quad q_1 = 1, \quad r_1 = 2.$$

Por tanto, de acuerdo a nuestro algoritmo, para calcular $d = \text{mcd}(6, 4)$, basta con calcular

$$d = \text{mcd}(4, 2) \quad 4 = 2(2) + 0, \quad q_2 = 2, \quad r_2 = 0.$$

Notemos que hemos usado nuevamente el algoritmo de la división para calcular los valores de q_2 y r_2 . Finalmente, usando recursión nuevamente, obtenemos que

$$d = \text{mcd}(2, 0) = 2.$$

2. En este ejemplo tomaremos $n = 32$ y $m = 24$. Como el razonamiento es muy similar al del ejemplo anterior, escribiremos solamente los cálculos necesarios para obtener el resultado correspondiente:

$$\begin{aligned} d &= \text{mcd}(32, 24) & q_1 &= 1, \quad r_1 = 8 \\ &= \text{mcd}(24, 8) & q_2 &= 3, \quad r_2 = 0 \\ &= \text{mcd}(8, 0) \\ &= 8 \end{aligned}$$

3. Finalmente consideremos un ejemplo un poco mas complicado que los anteriores: $n = 37$, $m = 14$.

$$\begin{aligned}
 d &= \text{mcd}(37, 14) & q_1 &= 1, & r_1 &= 8 \\
 &= \text{mcd}(14, 9) & q_2 &= 1, & r_2 &= 5 \\
 &= \text{mcd}(9, 5) & q_3 &= 1, & r_3 &= 4 \\
 &= \text{mcd}(5, 4) & q_4 &= 1, & r_4 &= 1 \\
 &= \text{mcd}(4, 1) & q_5 &= 1, & r_5 &= 0 \\
 &= \text{mcd}(1, 0) \\
 &= 1
 \end{aligned}$$

Regresemos ahora a la demostración del teorema. Hasta ahora hemos proporcionado un algoritmo para calcular el valor de

$$d = \text{mcd}(n, m).$$

De acuerdo con la interpretación BHK, ahora debemos demostrar que el número calculado satisface las condiciones requeridas, es decir, debemos demostrar que:

1. $d > 0$.
2. $d|n$ y $d|m$.
3. Si $t|n$ y $t|m$, entonces $t|d$.

Empecemos con la primera propiedad:

1. **$d > 0$.** Para demostrar esta propiedad, observemos primero que el teorema requiere que $n > m \geq 0$. Si $m = 0$, entonces

$$d = \text{mcd}(n, m) = n > 0$$

por hipótesis. Si $m \neq 0$, entonces, como $m \geq 0$ obtenemos que $m > r_1 \geq 0$, donde

$$n = q_1 m + r_1$$

y ahora buscamos calcular

$$d = \text{mcd}(m, r_1).$$

Notemos que en cada paso del algoritmo la primera entrada es un número estrictamente positivo y en el último paso simplemente extraemos este número que, por tanto, deberá de ser estrictamente positivo.

2. **$d|n$ y $d|m$.** Para demostrar esta propiedad, analizaremos la estructura del algoritmo de manera

mas detenida:

$$\begin{array}{lll}
\text{mcd}(n, m) & n = q_1 m + r_1, & r_1 = n - q_1 m. \\
\text{mcd}(m, r_1) & m = q_2 m + r_2, & r_2 = m - q_2 m. \\
\text{mcd}(r_1, r_2) & r_1 = q_3 m + r_3, & r_3 = r_1 - q_3 m. \\
\vdots & \vdots & \vdots \\
\text{mcd}(r_{k-2}, r_{k-1}) & r_{k-2} = q_k r_{k-1} + r_k, & r_k = r_{k-2} - q_k r_{k-1}. \\
\text{mcd}(r_{k-1}, r_k) & r_{k-1} = q_{k+1} r_k + r_{k+1}, & r_{k+1} = r_{k-1} - q_{k+1} r_k. \\
\text{mcd}(r_k, r_{k+1}) & r_k = q_{k+2} r_{k+1}, & 0 = r_k - q_{k+2} r_{k+1}. \\
\text{mcd}(r_{k+1}, 0) & & \\
r_{k+1} & &
\end{array}$$

Notamos que en el paso $k + 1$ tenemos la ecuación

$$r_k = q_{k+2} r_{k+1}.$$

Este es el penúltimo paso del algoritmo y notamos que al dividir r_k por r_{k+1} no obtenemos residuo, es decir, $r_{k+1} | r_k$. Notemos también que $r_{k+1} | r_{k+1}$ trivialmente. Por otro lado, de la ecuación

$$r_{k-1} = q_{k+1} r_k + r_{k+1}$$

obtenemos que $r_{k+1} | r_{k-1}$. Pero ahora, como ya sabíamos que $r_{k+1} | r_k$ y como

$$r_{k-2} = q_k r_{k-1} + r_k,$$

obtenemos que $r_{k+1} | r_{k-2}$. Prosiguiendo de esta manera, obtenemos que

$$r_{k+1} | r_{k-2}, \dots, r_{k+1} | r_1, r_{k+1} | m, r_{k+1} | n.$$

En particular, hemos demostrado que, si definimos $d = \text{mcd}(n, m)$, entonces

$$d = r_{k+1} | m \quad \text{y} \quad d | n.$$

3. **Si $t | n$ y $t | m$, entonces $t | d$.** Ahora supongamos que existe $t \in \mathbb{Z}$ tal que $t | n$ y $t | m$. Queremos demostrar que, en este caso, $t | d$. Empecemos por observar que

$$t | n - q_1 m = r_1.$$

Pero entonces, por el mismo argumento, obtenemos que

$$t | m - q_2 r_1 = r_2.$$

Repitiendo este argumento de manera repetida, obtenemos, finalmente que

$$t | r_{k-1} - q_{k+1} r_k = r_{k+1} = d,$$

tal y como queríamos demostrar.

Ahora modificaremos el algoritmo dado anteriormente para obtener una nueva función, que llamaremos emcd , tal que si

$$(a, b, d) = \text{emcd}(n, m),$$

entonces

$$d = \text{mcd}(n, m) \quad \text{y} \quad d = an + bm.$$

El algoritmo es el siguiente:

1. Si $m = 0$, definimos

$$(a, b, d) = \text{emcd}(n, m) = (1, 0, n).$$

2. Si $m \neq 0$, entonces, usando el algoritmo de la división, encontramos q, r , con $0 \leq r < m$ tales que

$$n = qm + r.$$

En este caso, definimos

$$(a, b, d) = (\text{emcd}(m, r)_2, \text{emcd}(m, r)_1 - q \text{emcd}(m, r)_2, \text{emcd}(m, r)_3),$$

donde $\text{emcd}(m, r)_1$, $\text{emcd}(m, r)_2$ y $\text{emcd}(m, r)_3$ representan la primera, segunda y tercera entrada, respectivamente de la tripleta asociada a $\text{emcd}(m, r)$.

Nuevamente, ilustraremos el funcionamiento del algoritmo mediante el análisis de algunos casos particulares:

1. Cuando $n = 6$, $m = 4$, obtenemos:

$$\begin{aligned} q_1 = 1, \quad r_1 = 2, \quad & \text{emcd}(6, 4) = (1, 0 - (1)(1), 2) = (1, -1, 2) \\ q_2 = 2, \quad r_2 = 0, \quad & \text{emcd}(4, 2) = (0, 1 - (2)(0), 2) = (0, 1, 2) \\ & \text{emcd}(2, 0) = (1, 0, 2) \end{aligned}$$

2. Cuando $n = 32$, $m = 24$, obtenemos:

$$\begin{aligned} q_1 = 1, \quad r_1 = 8, \quad & \text{emcd}(32, 24) = (1, 0 - (1)(1), 8) = (1, -1, 8) \\ q_2 = 3, \quad r_2 = 0, \quad & \text{emcd}(24, 8) = (0, 1 - (3)(0), 8) = (0, 1, 8) \\ & \text{emcd}(8, 0) = (1, 0, 8) \end{aligned}$$

3. Cuando $n = 37$, $m = 14$ obtenemos:

$$\begin{aligned} q_1 = 2, \quad r_1 = 9, \quad & \text{emcd}(37, 14) = (-3, 2 - (2)(-3), 1) = (-3, 8, 1) \\ q_2 = 1, \quad r_2 = 5, \quad & \text{emcd}(14, 9) = (2, -1 - (1)(2), 1) = (2, -3, 1) \\ q_3 = 1, \quad r_3 = 4, \quad & \text{emcd}(9, 5) = (-1, 1 - (1)(-1), 1) = (-1, 2, 1) \\ q_4 = 1, \quad r_4 = 1, \quad & \text{emcd}(5, 4) = (1, 0 - (1)(1), 1) = (1, -1, 1) \\ q_5 = 4, \quad r_5 = 0, \quad & \text{emcd}(4, 1) = (0, 1 - (4)(0), 1) = (0, 1, 1) \\ & \text{emcd}(1, 0) = (1, 0, 1) \end{aligned}$$

Lo único que nos queda por demostrar es que si $(a, b, d) = \text{emcd}(n, m)$ entonces la identidad $d = an + bm$ se satisface. Para demostrar esta identidad usaremos inducción:

Caso base Supongamos que $m = 0$. En este caso

$$\text{emcd}(n, 0) = (1, 0, n)$$

y, claramente,

$$d = an + bm = 1(n) + 0(0) = n.$$

Paso Inductivo Supongamos que

$$\text{emcd}(r_j, r_{j+1}) = (a_j, b_j, r_{k+1}),$$

es decir, supongamos que

$$a_j r_j + b_j r_{j+1} = r_{k+1}. \quad (1)$$

Queremos demostrar que

$$\text{emcd}(r_{j-1}, r_j) = (b_j, a_j - q_{j+1}b_j, r_{k+1}),$$

en otras palabras, queremos demostrar que

$$r_{k+1} = b_j r_{j-1} + (a_j - q_{j+1}b_j) r_j.$$

Pero, como $r_{j-1} = q_{j+1}r_j + r_{j+1}$, tenemos que

$$\begin{aligned} b_j r_{j-1} + (a_j - q_{j+1}b_j) r_j &= b_j (q_{j+1}r_j + r_{j+1}) + (a_j - q_{j+1}b_j) r_j \\ &= b_j q_{j+1} r_j + b_j r_{j+1} + a_j r_j - q_{j+1} b_j r_j \\ &= a_j r_j + b_j r_{j+1} \\ &= r_{k+1}, \end{aligned}$$

donde en el último paso hemos usado la ecuación (1).

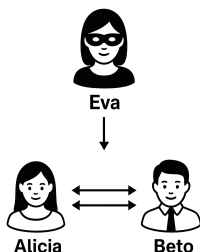
□

2 Protocolos de encriptación clásicos

En esta sección consideraremos los protocolos criptográficos clásicos, es decir, aquellos basados en aritmética modular y que no utilizan curvas elípticas.

2.1 El protocolo de Diffie-Hellman

Supongamos que tenemos dos agentes, digamos Alicia y Beto, que desean comunicarse a través de un canal inseguro. Supongamos además que tenemos un agente hostil, digamos Eva, que puede interceptar toda la comunicación entre Alicia y Beto.



El objetivo del protocolo de Diffie-Hellman es que Alicia y Beto puedan acordar una llave secreta, la cual podrá ser utilizada posteriormente para cifrar la comunicación entre ellos utilizando un algoritmo de cifrado simétrico como AES.

Los pasos del protocolo son los siguientes:

1. Alicia y Beto acuerdan un número primo p y una base g . Estos números pueden ser públicos y conocidos por todos los agentes, incluyendo a Eva.
2. Alicia y Beto eligen llaves privadas a y b respectivamente, las cuales son números enteros aleatorios menores que p . Estas llaves son secretas y no deben ser compartidas con nadie.
3. Alicia y Beto calculan sus llaves públicas A y B respectivamente, utilizando la siguiente fórmula:

$$A = g^a \mod p$$

para la llave pública de Alicia y

$$B = g^b \mod p$$

para la llave pública de Beto. Estas llaves públicas son enviadas a través del canal inseguro y pueden ser interceptadas por Eva.

4. Alicia y Beto calculan la llave secreta compartida s utilizando la siguiente fórmula:

$$s_A = B^a = (g^b)^a = g^{ba} \mod p$$

para Alicia y

$$s_B = A^b = (g^a)^b = g^{ab} \mod p$$

para Beto. Notamos que $s_A = s_B$, es decir, ambos agentes han acordado la misma llave secreta.

Nota: Para elegir los parámetros p y g es común utilizar una de las opciones listadas en la especificación [RFC 3526](#) publicada por el IETF en mayo de 2003. Esta especificación incluye una lista de números primos de 1536, 2048, 3072, 4096, 6144 y 8192 bits, así como sus respectivas bases.

En general, un *primo de Sophie Germain* es un número primo q tal que $p = 2q + 1$ también es primo. En este caso, decimos que p es un *primo seguro*. Todos los primos de la lista de la especificación RFC 3526 son primos seguros. En este caso, para evitar ataques de subgrupo, se recomienda que las llaves públicas A y B pertenezcan al subgrupo de orden g generado por g^2 . Para esto basta generar números aleatorios $\tilde{a}, \tilde{b} \leq q - 1$ y tomar $a = 2\tilde{a}$ y $b = 2\tilde{b}$. En general, para asegurar que el protocolo es resistente a ataques por fuerza bruta, se recomienda que $\tilde{a}$ y $\tilde{b}$ sean números aleatorios de al menos 256 bits.

2.2 El sistema RSA

El sistema RSA, introducido por Rivest, Shamir y Adleman en 1977, es un sistema criptográfico de clave pública que se utiliza ampliamente en la industria. Este sistema consiste de cinco algoritmos:

1. Un algoritmo para la generación de una *llave pública* y una *llave privada*.
2. Un algoritmo para el *cifrado* de mensajes.
3. Un algoritmo para el *descifrado* de mensajes.
4. Un algoritmo para la *firma* de mensajes.
5. Un algoritmo para la *verificación* de firmas.

2.2.1 Algoritmo RSA para la generación de llaves

El algoritmo para la generación de las llaves pública y privada consiste de los siguientes pasos:

1. Alicia selecciona dos números primos aleatorios p y q .
2. Alicia calcula

$$n = pq, \quad \phi(n) = (p - 1)(q - 1).$$

3. Alicia elige un número $0 < e < \phi(n)$ tal que

$$\text{mcd}(e, \phi(n)) = 1$$

4. Alicia calcula un $1 < d < \phi(n)$ tal que

$$ed = 1 \quad \text{mod } \phi(n)$$

5. Alicia define su llave pública y privada como

$$\text{llave_publica} = (n, e)$$

$$\text{llave_privada} = (n, d).$$

y comunica su llave pública a Beto.

Notas: Con respecto al algoritmo para la generación de llaves, notemos lo siguiente:

1. Eva también puede conocer la llave pública de Alicia, pero no su llave privada.
2. En la práctica, el número e es comúnmente elegido como $e = 3$ o $e = 65537$. Estos números son elegidos por razones de eficiencia, ya que permiten realizar el cifrado de manera más rápida. Sin embargo, es importante asegurarse de que $\text{mcd}(e, \phi(n)) = 1$.
3. En la práctica, el número d es calculado utilizando el algoritmo de Euclides extendido, el cual permite encontrar el inverso multiplicativo de e módulo $\phi(n)$. Este algoritmo es eficiente y se puede implementar fácilmente en cualquier lenguaje de programación.
4. En la práctica, el número d es comúnmente almacenado en un formato binario, lo cual permite una representación más compacta y eficiente. Sin embargo, es importante asegurarse de que el número d sea almacenado de manera segura y no sea accesible a agentes no autorizados.
5. Notemos que si logramos factorizar n en sus factores primos p y q , entonces podemos calcular $\phi(n)$ y, por lo tanto, podemos calcular la llave privada d . Por esta razón, la seguridad del sistema RSA depende de la dificultad de factorizar números enteros grandes. En general, se recomienda utilizar números primos de al menos 1024 bits para asegurar la integridad del sistema.

2.2.2 Algoritmo RSA para el cifrado de mensajes

El algoritmo consiste de los siguientes pasos:

1. Beto selecciona un mensaje m_b tal que $0 < m_b < n$ y define

$$\text{mensaje_llano} = (m_b, n)$$

2. Beto calcula

$$c = m_b^e \mod n.$$

y define

$$\text{mensaje_cifrado} = (c, n).$$

Este es el mensaje cifrado que Beto puede compartir con Alicia a través del canal inseguro.

2.2.3 Algoritmo RSA para el descifrado de mensajes

El algoritmo consiste de los siguientes pasos:

1. Alicia calcula

$$\tilde{m}_b = c^d \mod n$$

y define

$$\text{mensaje_descifrado} = (\tilde{m}_b, n).$$

2. Alicia puede proceder ahora a leer el mensaje descifrado $\tilde{m}_b$.

Notas:

1. Notemos que como Eva no conoce la llave privada de Alicia, no puede descifrar el mensaje cifrado c . Por lo tanto, el mensaje cifrado es seguro y no puede ser leído por Eva.
2. En la práctica, el sistema de encriptación RSA es mucho más ineficiente que el uso de algoritmos de cifrado simétrico como AES. Como vimos en la sección anterior, el algoritmo de Diffie-Hellman permite acordar una llave secreta entre dos agentes, la cual puede ser utilizada posteriormente para cifrar la comunicación entre ellos. Es por eso que este algoritmo ya no es tan utilizado en la práctica. Sin embargo, el algoritmo RSA sigue siendo ampliamente utilizado para la firma digital de mensajes y la verificación de firmas.

2.2.4 Algoritmo RSA para la firma de mensajes

El algoritmo consiste de los siguientes pasos:

1. Alicia selecciona un mensaje m_a tal que $0 < m_a < n$ y define

$$\text{mensaje_llano} = (m_a, n).$$

2. Alicia utiliza una función de hash para calcular el hash del mensaje m_a , es decir, calcula

$$h = \text{Hash}(m_a).$$

3. Alicia calcula

$$s = h^d \mod n$$

y define

$$\text{mensaje_firmado} = (m_a, s).$$

Este es el mensaje firmado que Alicia puede compartir con Beto a través del canal inseguro.

2.2.5 Algoritmo RSA para la verificación de firmas

El algoritmo consiste de los siguientes pasos:

1. Beto utiliza una función de hash para calcular el hash del mensaje m_a , es decir, calcula

$$h = \text{Hash}(m_a).$$

2. Beto calcula

$$\tilde{h} = s^e \quad \text{mod } n$$

y verifica si

$$h = \tilde{h} \quad \text{mod } n.$$

Notas: En la práctica, el hash h es comúnmente calculado utilizando una función de hash como SHA-256 o SHA-512, lo cual permite una representación más compacta y eficiente. Sin embargo, es importante asegurarse de que la función de hash utilizada sea segura y no sea vulnerable a ataques de colisión.

En general los algoritmos de generación de firma digital y su verificación buscan cumplir con los siguientes objetivos:

1. **Autenticidad:** El receptor del mensaje debe poder verificar que el mensaje fue realmente enviado por el remitente y no por un agente malicioso. Esto se logra utilizando la llave privada del remitente para firmar el mensaje y la llave pública del remitente para verificar la firma. Como solamente Alicia tiene acceso a su llave privada, solamente ella puede firmar el mensaje.
2. **Integridad:** El receptor del mensaje debe poder verificar que el mensaje no ha sido modificado durante la transmisión. Esto se logra utilizando una función de hash para calcular el hash del mensaje y compararlo con el hash firmado por el remitente. Si los hashes son iguales, entonces el mensaje no ha sido modificado.
3. **No repudio:** El remitente del mensaje no puede negar haber enviado el mensaje. Esto se logra utilizando la llave privada del remitente para firmar el mensaje y la llave pública del remitente para verificar la firma. Como solamente Alicia tiene acceso a su llave privada, solamente ella puede firmar el mensaje y por tanto no puede negar haber enviado el mensaje ni negar el contenido del mismo.

Ejemplo 2.1. Ilustraremos ahora los algoritmos del protocolo RSA con ejemplos concretos.

1. **Generación de llaves.** Tomaremos

$$p = 5, \quad q = 11.$$

Entonces tendremos que

$$n = 5(11) = 55, \quad \phi(n) = 4(10) = 40.$$

Elegimos ahora $e = 3$ y observamos que

$$\text{mcd}(3, 40) = 1.$$

Utilizando el algoritmo de Euclides extendido, calculamos el inverso de 3 módulo 40 para obtener

$$d = 27.$$

Por lo tanto, tenemos que

$$\begin{aligned}\text{llave_publica} &= (55, 3) \\ \text{llave_privada} &= (55, 27).\end{aligned}$$

2. **Cifrado de mensajes.** Supongamos que Beto quiere enviar el mensaje $m_b = 7$ a Alicia. Entonces, Beto calcula

$$c = 7^3 = 13 \quad \text{mod } 55.$$

Por lo tanto, el mensaje cifrado que Beto envía a Alicia es

$$\text{mensaje_cifrado} = (13, 55).$$

3. **Descifrado de mensajes.** Alicia recibe el mensaje cifrado $c = 13$ y calcula

$$\tilde{m}_b = 13^{27} = 7 \quad \text{mod } 55.$$

Notemos que el mensaje descifrado es el mismo que el mensaje original enviado por Beto. Por lo tanto, Alicia puede leer el mensaje descifrado $\tilde{m}_b = 7$.

4. **Firma de mensajes.** Supongamos que Alicia quiere firmar el mensaje $m_a = 9$. Entonces, Alicia define

$$\text{mensaje_sin_firmar} = (9, 55).$$

Para este ejemplo, consideraremos la función de hasheo como la función identidad, es decir,

$$h = \text{Hash}(m_a) = m_a = 9.$$

Por tanto Alicia debe de calcular

$$s = 9^{27} = 4 \quad \text{mod } 55.$$

Se sigue que el mensaje firmado que Alicia envía a Beto es

$$\text{mensaje_firmado} = (9, 4).$$

5. **Verificación de firmas.** Beto recibe el mensaje firmado $(9, 4)$ y calcula

$$\tilde{h} = 4^3 = 9 \quad \text{mod } 55.$$

Beto verifica que el mensaje firmado es correcto al comprobar que

$$\tilde{h} = 9 = h \quad \text{mod } 55.$$

2.2.6 Detalles técnicos del algoritmo RSA

Para aplicar y entender el algoritmo RSA, debemos resolver los siguientes problemas técnicos:

1. En el paso 4 del algoritmo RSA para la generación de llaves debemos resolver la ecuación

$$ed \equiv 1 \pmod{\phi(n)}$$

bajo la condición de que $\text{mcd}(e, \phi(n)) = 1$. Queremos encontrar un algoritmo eficiente para calcular d .

2. Demostrar que, efectivamente,

$$\tilde{m} = c^d \equiv (m^e)^d \equiv m^{ed} = m \pmod{n}.$$

3. Estimar la seguridad del sistema. En particular, tratar de estimar el tiempo que tomaría romper el sistema cuando se desconoce la llave privada.

Observación 2.2. En la práctica, los valores de p y q deben ser muy grandes para asegurar la integridad del sistema. Por ejemplo, si tomamos $p = 5$ y $q = 3$, entonces la siguiente tabla nos da la información necesaria para romper el sistema:

m	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
c	0	1	8	12	4	5	6	13	12	9	10	11	3	7	14